

EE 475 Introduction to Image Processing
Fall 2025 Final Project Report

Multi-Modal Image Processing Pipeline for Automated Glioma Segmentation

A Classical Approach to the BraTS 2021 Challenge

Boran Aybak Kılıç - 2022401207
Aziz Umut Bakırcı - 2023401309

Word count $\approx$ 1600

July 6, 2026

1 Introduction

Glioblastoma Multiforme (GBM) represents the most aggressive and heterogeneous class of primary brain tumors. The prognosis for patients is often poor, making early diagnosis and precise treatment planning critical for survival. Accurate segmentation of tumor sub-regions—including the necrotic core (NCR), peritumoral edema (ED), and enhancing tumor (ET)—is essential for quantifying tumor volume and guiding radiotherapy. While Magnetic Resonance Imaging (MRI) is the standard modality for diagnosis, manual segmentation is time-consuming and subject to inter-observer variability, necessitating automated solutions.

Historically, brain tumor segmentation has evolved from simple manual delineation to semi-automatic thresholding and region-growing techniques [1]. Early conventional methods utilized thresholding (global and local) and region-based techniques like Watershed and Region Growing, though they often struggled with intensity inhomogeneity and the complex, irregular boundaries of gliomas [1]. While recent advancements have shifted heavily toward Deep Learning (e.g., U-Nets), these models require massive annotated datasets and often lack interpretability.

In this project, we propose a robust, fully automated pipeline utilizing classical image processing. Our contribution lies in the intelligent combination of morphological operations, geometric constraints, and physics-based contrast enhancement. Unlike "black-box" neural networks, our method iteratively restricts the segmentation domain—moving from the Whole Tumor (WT) to the Tumor Core (TC) and finally the Necrotic Core (NCR)—utilizing the biological hierarchy of glioma layers ($NCR \subset ET \subset WT$). By explicitly modeling the geometry of the tumor (e.g., using convex hulls for seed constraints), we achieve decent segmentation without the need for training data.

2 Materials and Methods

2.1 Dataset and Preprocessing

We utilized the <https://www.kaggle.com/datasets/dschettler8845/brats-2021-task1>, consisting of multi-modal MRI scans (T1, T1CE, T2, FLAIR) from 1251 patients. To resolve intensity inhomogeneity and standardize the data, we applied the following preprocessing sequence:

1. **Brain Mask Extraction:** for preventing the bias of background pixels and restricting our region of interest.
2. **N4 Bias Field Correction** to remove low-frequency intensity variations.
3. **Z-score Normalization** to standardize intensity ranges.
4. **Gaussian Smoothing** ($\sigma = 1.0$) to reduce high-frequency thermal noise.

2.2 Segmentation Pipeline

Our approach decomposes the segmentation problem into three hierarchical stages.

2.2.1 Whole Tumor (WT) Extraction

The Whole Tumor encompasses both the tumor core. We extract the WT mask (M_{WT}) using a percentile-based thresholding approach followed by morphological refinement to remove noise:

$$M_{init} = I_{FLAIR} > P_{50}(I_{FLAIR}) \quad (1)$$

To ensure topological consistency, we apply morphological opening with a large radius to detach the tumor from skull artifacts, select the largest connected component (CC_{max}), and apply closing to fill gaps:

$$M_{WT} = CC_{max}((M_{init} \circ B_{ball}) \bullet B_{ball}) \quad (2)$$

This mask M_{WT} serves as the global hierarchical constraint for all subsequent steps.

2.2.2 Enhancing Tumor (ET) Extraction

The enhancing tumor indicates active malignancy and is characterized by contrast uptake. We amplify this signal by computing an enhancement map (I_{enh}) utilizing the T1CE and native T1 modalities:

$$I_{enh} = I_{T1CE} - I_{T1} \quad (3)$$

Since cerebrospinal fluid (CSF) and blood vessels can also appear bright, we perform two critical filtering steps. First, we compute an adaptive Otsu threshold (τ_{otsu}) strictly considering pixels within the M_{WT} region:

Let $I : \Omega \rightarrow \{0, 1, \dots, L-1\}$ be a grayscale image, $R \subseteq \Omega$ the MWT (ROI) mask, and $W(x) \subseteq \Omega$ a local window centered at pixel $x \in \Omega$. We define the local masked region

$$S(x) = R \cap W(x).$$

The region restricted Otsu threshold is then

$$\tau(x) = \tau_{otsu}(S(x)).$$

so that $\sum_{i=0}^{L-1} p_S(i) = 1$. For a candidate threshold $t \in \{0, 1, \dots, L-2\}$, define class probabilities

$$\omega_0(t) = \sum_{i=0}^t p_S(i), \quad \omega_1(t) = \sum_{i=t+1}^{L-1} p_S(i) \quad (4a)$$

$$\mu_0(t) = \frac{1}{\omega_0(t)} \sum_{i=0}^t i p_S(i), \quad \mu_1(t) = \frac{1}{\omega_1(t)} \sum_{i=t+1}^{L-1} i p_S(i) \quad (4b)$$

$$\sigma_B^2(t) = \omega_0(t) \omega_1(t) (\mu_0 - \mu_1)^2 \quad (4c)$$

$$\tau_{otsu}(S) = \arg \max_{t \in \{0, \dots, L-2\}} \sigma_B^2(t) \quad (4d)$$

2.2.3 Necrotic Core (NCR) Extraction via Seeded Region Growing and Watershed

The necrotic core (NCR) typically resides within the internal volume of the enhancing tumor ring [2]. To segment this region, we utilize Seeded Region Growing (SRG). A

primary challenge in SRG is the automated initialization of seed points [3]. We address this by computing the 3D Convex Hull of the ET mask, $CH(M_{ET})$, to define a restricted geometric search domain.

Seeds S are identified by evaluating $N \times N \times N$ volumetric blocks and selecting those with the highest mean intensity within $CH(M_{ET})$. The region R expands iteratively by incorporating adjacent voxels x that satisfy the following relative homogeneity predicate [2]:

$$|I(x) - \mu_R| < \theta \cdot \mu_R \quad (5)$$

where μ_R represents the running mean of the growing region and θ is a tunable threshold coefficient.

Alternatively, for lesions with poorly defined boundaries, we implement a Marker-Controlled Watershed algorithm [4] initialized with these same seed points. To construct the topographical surface for flooding, we compute the morphological gradient $G(I)$. This operation enhances class transitions while maintaining robustness against noise compared to standard gradient operators [5]:

$$G(I) = (I \oplus B) - (I \ominus B) \quad (6)$$

where $\oplus$ and $\ominus$ denote dilation and erosion by a 3D structuring element B . This gradient map defines the ridges that serve as the final segmentation boundaries [4].

2.3 Performance Metrics

The pipeline performance is quantified using the Dice Similarity Coefficient (DSC) and the 95th percentile Hausdorff Distance (HD_{95}) against the BraTS ground truth [6].

2.3.1 Dice Similarity Coefficient (DSC)

The DSC evaluates the volumetric spatial overlap between the predicted segmentation P and the ground truth G :

$$DSC(P, G) = \frac{2|P \cap G|}{|P| + |G|} \quad (7)$$

This metric ranges from 0 to 1, where 1 signifies perfect alignment.

2.3.2 95th Percentile Hausdorff Distance (HD_{95})

The Hausdorff distance computes the maximum distance between the surface of the predicted region and the ground truth. We employ the 95th percentile (HD_{95}) specifically because it is significantly more resilient to small outliers and human labeling inconsistencies than the standard Hausdorff distance:

$$HD(P, G) = \max(h(P, G), h(G, P)) \quad (8)$$

where $h(P, G)$ represents the directed Hausdorff distance between point sets P and G .

3 Results

The proposed pipeline was tested on 12 subjects from the BraTS validation set. The pre-processing stage effectively standardized the volumetric data, removing non-brain tissues and normalizing intensity ranges to ensure consistent thresholding across the cohort.

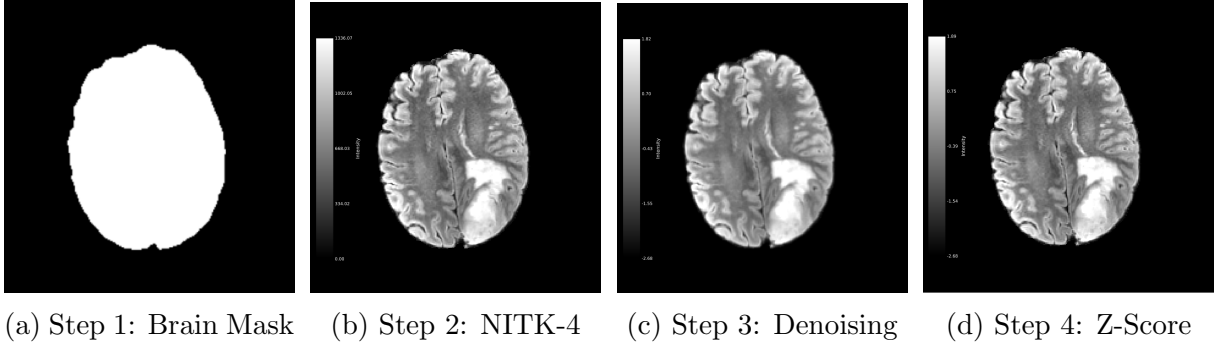


Figure 1: Preprocessing Steps Shown in One Slice of FLAIR Image

Following preprocessing, the Whole Tumor (WT) extraction relied on morphological consistency. Figure 2 demonstrates how 3D connected component analysis was critical in discarding disjoint noise artifacts that appeared significant in 2D slices but lacked volumetric continuity.

For the Enhancing Tumor (ET), the difference map ($T1CE - T1$) provided a distinct bimodal histogram for most patients, allowing Otsu’s method to automatically identify the optimal separation threshold. As shown in Figure 3, enforcing the spatial hierarchy ($ET \subset WT$) eliminated false positive enhancements located outside the main tumor volume.

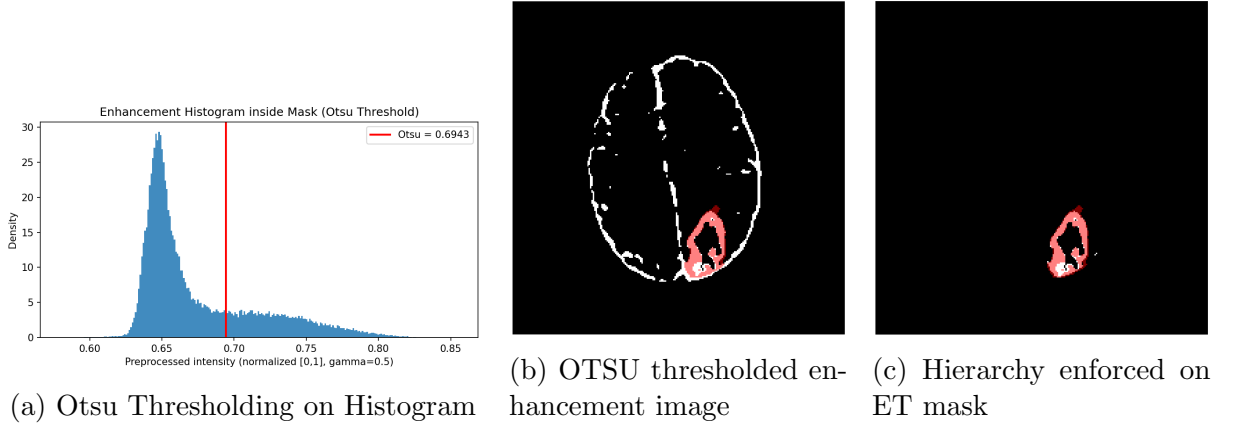


Figure 3: Enhancement image histogram and ET extraction process.

The segmentation of the Necrotic Core (NCR) proved to be the most complex task. Figure 4 illustrates the successful containment of the region growing process; the Convex Hull successfully prevented the algorithms from leaking into non-tumor dark regions, such as the ventricles.

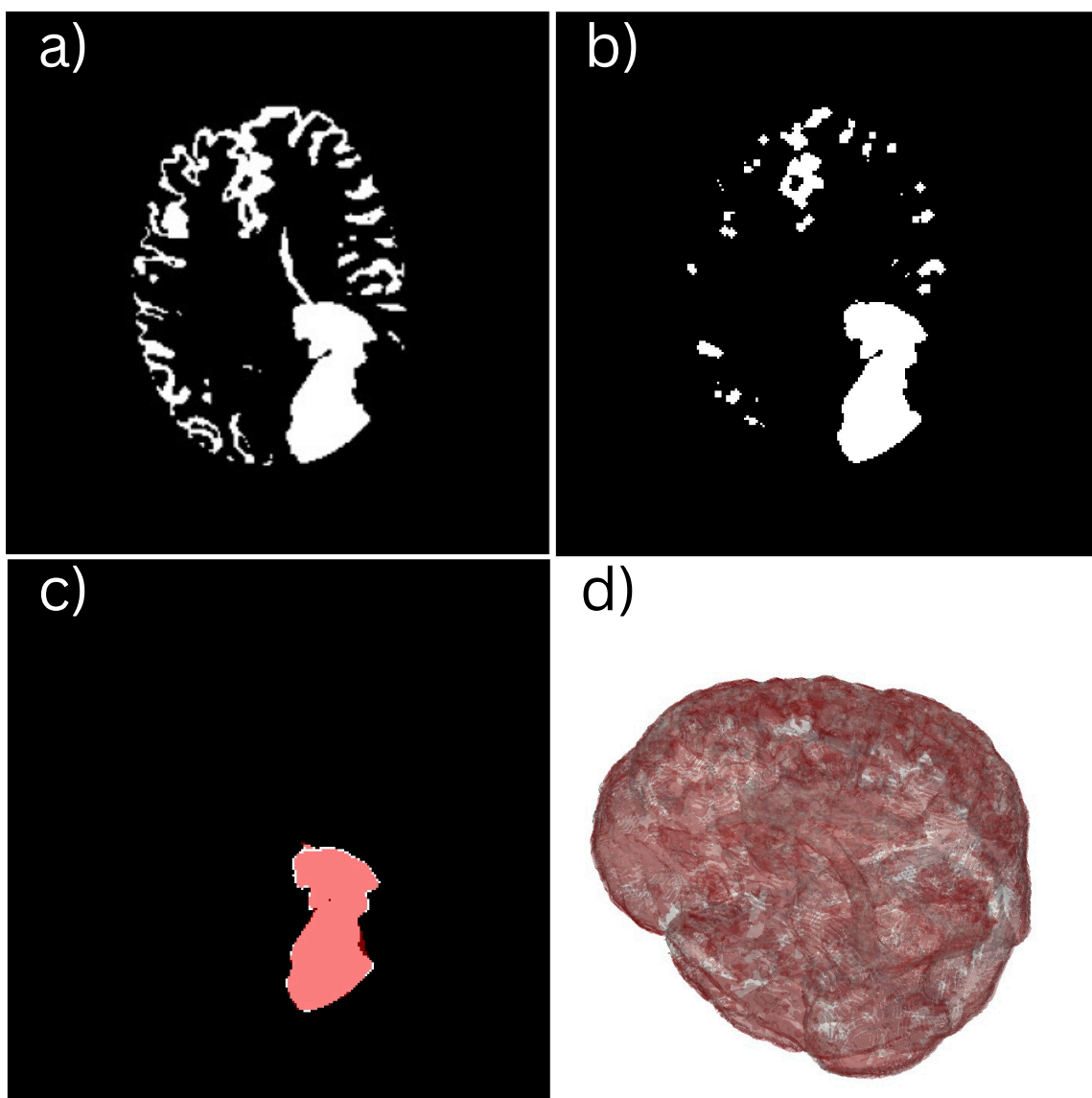


Figure 2: Whole Tumor Extraction steps: a) 50% thresholded FLAIR, b) Opening with radius 5 spherical structuring element. c) Selecting Largest component and dilation, d) 3D connectivity of the brain which is not apparent in 2D

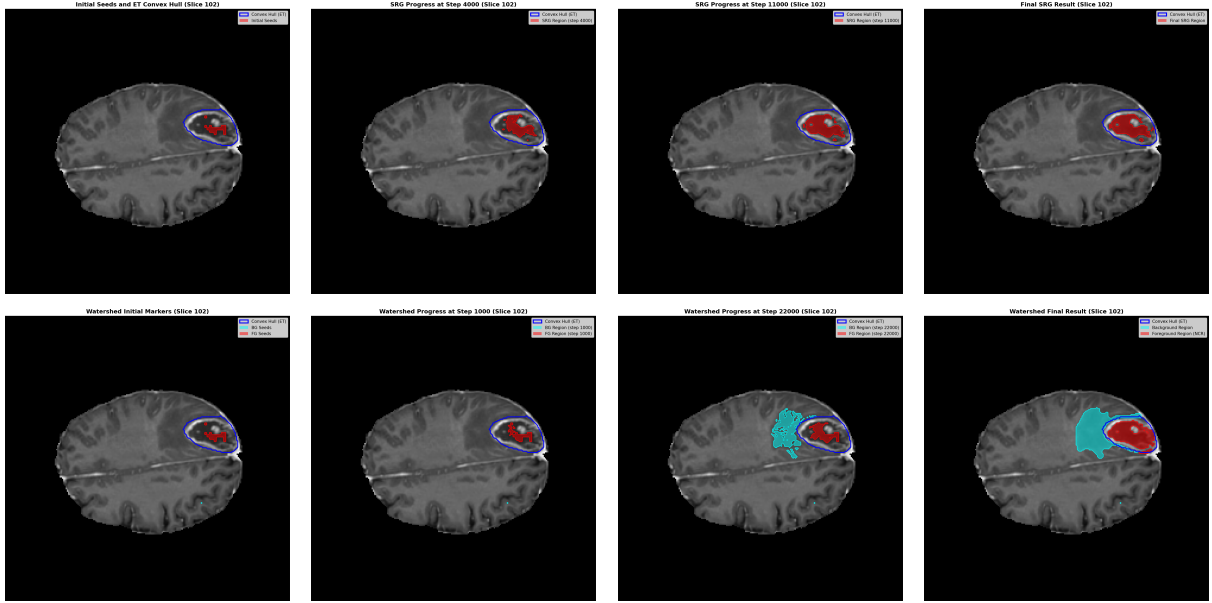


Figure 4: Progression of Seed Region Growing (top row) and Watershed segmentation (bottom row). Red points indicate foreground seeds, blue contours represent the convex hull, and light blue pixels denote the background regions in the watershed transformation.

Quantitative evaluation (Table 1) reveals strong performance for the Whole Tumor (0.839) and Tumor Core (0.846), validating the robustness of the global thresholding and morphological approach. However, the lower scores and higher standard deviations for Edema and Necrotic Core reflect the difficulty in defining boundaries for regions with subtle intensity gradients or irregular shapes.

Table 1: Quantitative Performance Metrics (Mean $\pm$ Std)

Region	Dice Score (DSC)	Hausdorff (HD_{95})
Whole Tumor (WT)	0.839 ± 0.085	11.11 ± 15.26 mm
Enhancing Tumor (ET)	0.802 ± 0.091	6.27 ± 11.54 mm
Necrotic Core (NCR)	0.742 ± 0.178	7.31 ± 9.70 mm
Tumor Core (TC)	0.846 ± 0.086	7.54 ± 12.17 mm
Edema (ED)	0.688 ± 0.150	14.15 ± 15.39 mm

Despite the general success, the pipeline struggled with atypical cases. Figure 5 highlights instances where low inter-tissue contrast or significant acquisition artifacts led to under-segmentation or "leakage" into healthy tissue, accounting for the high variance observed in our metrics.

4 Discussion

Our results demonstrate that a classical image processing pipeline, when constrained by geometric priors, achieves competitive segmentation performance without the need for extensive training data. The high Dice Similarity Coefficient (DSC) for the Whole Tumor (WT) validates the efficacy of our morphological chain, specifically the integration of low-percentile thresholding, morphological opening, and largest connected component selection.

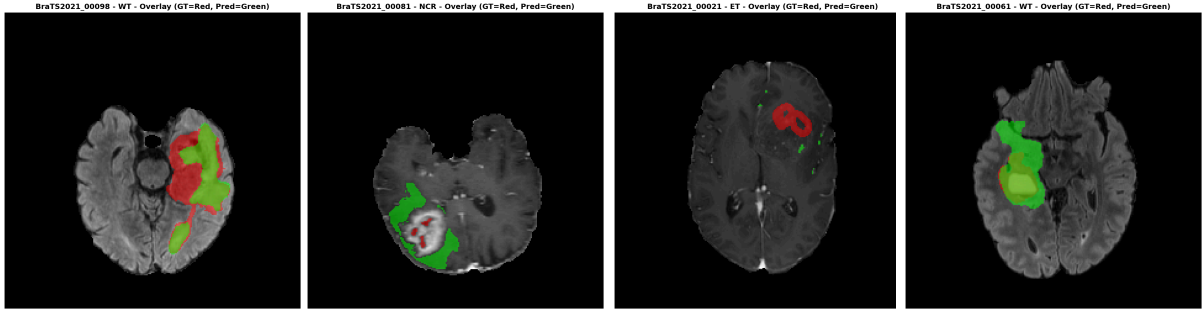


Figure 5: Examples of poor classifications and image quality

While our methodology initially contemplated slice-by-slice processing to leverage 2D-optimized algorithms, we ultimately transitioned to full 3D volumetric analysis to maintain global contextual features that 2D projections fail to capture. We observed that slice-wise processing often overlooks crucial inter-slice correlations, leading to a loss of global feature integrity. Standardizing on 3D operations—such as 3D Gaussian filtering and isotropic structuring elements—is fundamental for ensuring spatial continuity along the axial direction and preventing disconnected topologies across contiguous slices.

Our results demonstrate that a classical image processing pipeline, when constrained by geometric priors, achieves competitive segmentation performance without the need for extensive training data. The high Dice Similarity Coefficient (DSC) for the Whole Tumor (WT) validates the efficacy of our morphological chain, specifically the integration of low-percentile thresholding, morphological opening, and largest connected component selection.

However, the method faced limitations in scans exhibiting low inter-tissue contrast or significant acquisition artifacts. In several subjects, the intensity gradient between the WT boundaries and adjacent white matter was hardly present. Furthermore, the Necrotic Core (NCR) did not consistently exhibit the expected hypointensity on T1-weighted contrast-enhanced (T1CE) sequences, likely due to partial volume effects or heterogeneous tissue composition [6]. These data related challenges were compounded by inconsistencies in the ground truth; human labeling often lacked slice-to-slice continuity, resulting in jagged, non-biological "staircase" artifacts and inconsistent tissue classification [1].

Critically, the biological assumption of a strict spatial hierarchy ($NCR \subset TC \subset WT$) was not universally observed. We identified cases of multifocal glioblastoma where isolated tumor "islands," disjoint from the primary mass, were erroneously discarded by the largest component constraint. Additionally, automated seed selection for Seeded Region Growing (SRG) [2] and Watershed methods [4] proved difficult and sometimes obtained seeds were not reliable. While deriving a constraint region from the 3D Convex Hull of the Enhancing Tumor (ET) effectively localized seeds in typical cases [3], tumors with highly irregular or non-convex enhancing rings resulted in loose constraint regions. This geometric mismatch occasionally permitted seed placement in non-tumor tissue, leading to segmentation leakage.

5 Conclusion

We presented a fully automated, unsupervised pipeline for glioma segmentation driven by geometric logic. By rigorously enforcing the spatial hierarchy $NCR \subset ET \subset WT$, we successfully reduced the search space for ambiguous sub-regions. A primary contribu-

tion was the transition to a holistic 3D processing framework, which proved essential for ensuring axial continuity and capturing global pathological features that are lost in 2D slice-wise implementations.

Furthermore, the utilization of the ET-derived Convex Hull to constrain SRG seeds effectively eliminated false positives arising from distant dark regions, such as fluid pockets [3]. Our brain boundary subtraction method also successfully mitigated false ET detections caused by cortical CSF enhancement. Future work must focus on generalizing geometric assumptions to accommodate irregular glioblastoma topologies, optimizing thresholding parameters, and integrating geometric fallback mechanisms to ensure robustness in cases of gross misclassification.

References

- [1] Y. M. Mohammed, S. El Garouani, and I. Jellouli, “A survey of methods for brain tumor segmentation based mri images,” *Journal of Computational Design and Engineering*, vol. 9, 2022.
- [2] R. Adams and L. Bischof, “Seeded region growing,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 16, no. 6, pp. 641–647, 1994.
- [3] E. Biratu, F. Schwenker, T. Debelee, S. Kebede, W. Negera, and H. Molla, “Enhanced region growing for brain tumor mr image segmentation,” *Journal of Imaging*, vol. 7, no. 2, p. 22, 2021.
- [4] S. Ghaderi, S. Mohammadi, K. Ghaderi, F. Kiasat, and M. Mohammadi, “Marker-controlled watershed algorithm and fuzzy c-means clustering machine learning: automated segmentation of glioblastoma from mri images in a case series,” *Journal of Neuro-Oncology*, 2022, case Series.
- [5] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. Pearson, 2018.
- [6] U. Baid, S. Ghodasara, S. Mohan, M. Bilello, E. Calabrese, E. Colak, K. Farahani *et al.*, “The rsna-asnr-miccai brats 2021 benchmark on brain tumor segmentation and radiogenomic classification,” *arXiv preprint arXiv:2107.02314*, 2021.

A Python Implementation Code

Below is the simplified snippet of the core segmentation pipeline. There are many other utility, file-handling, visualization, edge-case handling and logging functions left out of the reporty. Full code repository can be accessed from <https://github.com/borankilic/brats>.

Listing 1: Hierarchical Segmentation Logic

```
1 # =====
2 # BraTS Tumor Segmentation Pipeline - Pseudocode (Default Methods)
3 # =====
4
5 # Libraries:
6 # - numpy (np): Array operations, mathematical functions
7 # - nibabel: NIfTI file I/O (load_nifti, save_nifti)
8 # - SimpleITK: N4ITK bias field correction
9 # - scipy.ndimage (ndi): Morphology, labeling, filtering
10 # - scipy.spatial: ConvexHull, Delaunay, directed_hausdorff
11 # - skimage.filters: threshold_otsu
12 # - skimage.exposure: adjust_gamma
13 # - skimage.morphology: ball, remove_small_objects
14 # - skimage.measure: regionprops
15 # - skimage.segmentation: watershed
16 # - json: Metrics serialization
17
18 def process_single_patient(patient_dir, output_dir=None):
19
20
21     # =====
22     # PHASE 1: DATA LOADING
23     # =====
24
25     # Load NIfTI files using nibabel
26     modalities = {}
27     patient_id = patient_dir.name
28
29     for modality in ['flair', 't1', 't1ce', 't2']:
30         file_path = patient_dir / f"{patient_id}_{modality}.nii.gz"
31         data, affine, header = load_nifti(file_path) # nibabel
32         modalities[modality] = (data, affine, header)
33
34     # =====
35     # PHASE 2: PREPROCESSING
36     # =====
37
38     # Step 2.1: Create Brain Mask (from FLAIR)
39     reference_data = modalities['flair'][0]
40
41     # Percentile thresholding (2nd percentile)
42     threshold = np.percentile(reference_data, 2.0) # numpy
43     brain_mask = reference_data > threshold
44
45     # Remove small objects
46     brain_mask = morphology.remove_small_objects(brain_mask, min_size
47         =1000) # skimage.morphology
48
49     # Fill holes
50     brain_mask = ndi.binary_fill_holes(brain_mask) # scipy.ndimage
```

```

50
51 # Gaussian smoothing
52 brain_mask = ndi.gaussian_filter(brain_mask.astype(float), sigma
    =1.0) > 0.5 # scipy.ndimage
53
54 # Keep largest connected component
55 labeled, num_features = ndi.label(brain_mask) # scipy.ndimage
56 if num_features > 0:
57     component_sizes = np.bincount(labeled.ravel()) # numpy
58     largest_label = component_sizes[1:].argmax() + 1
59     brain_mask = (labeled == largest_label)
60
61 # Step 2.2-2.4: Preprocess each modality
62 preprocessed = {}
63
64 for modality_name, (data, affine, header) in modalities.items():
65     processed = data.copy()
66
67     # Bias correction (N4ITK via SimpleITK)
68     processed = n4itk_bias_correction(
69         processed,
70         mask=brain_mask,
71         shrink_factor=4,
72         num_iterations=50
73     ) # SimpleITK.N4BiasFieldCorrectionImageFilter
74
75     # Z-score normalization
76     mean = np.mean(processed[brain_mask]) # numpy
77     std = np.std(processed[brain_mask]) # numpy
78     processed = (processed - mean) / (std + 1e-8)
79
80     # Gaussian filtering
81     processed = ndi.gaussian_filter(processed, sigma=1.0) # scipy.
        ndimage
82
83     preprocessed[modality_name] = processed
84
85 # =====
86 # PHASE 3: SEGMENTATION
87 # =====
88
89 masks = {}
90 flair = preprocessed['flair']
91 t1 = preprocessed['t1']
92 t1ce = preprocessed['t1ce']
93
94 # -----
95 # STEP 1: Segment Whole Tumor (WT)
96 # -----
97
98 # Threshold FLAIR at 50th percentile
99 flair_threshold = np.percentile(flair[brain_mask], 50.0) # numpy
100 flair_mask = (flair > flair_threshold) & brain_mask
101
102 # Threshold T2 at 50th percentile
103 t2 = preprocessed['t2']
104 t2_threshold = np.percentile(t2[brain_mask], 50.0) # numpy
105 t2_mask = (t2 > t2_threshold) & brain_mask

```

```

106
107 # Intersection of FLAIR and T2 masks
108 wt_mask = flair_mask & t2_mask # numpy
109
110 # Morphological operations
111 # Opening (radius=4)
112 se = morphology.ball(4) # skimage.morphology
113 wt_mask = ndi.binary_opening(wt_mask, structure=se) # scipy.ndimage
114
115 # Largest component
116 labeled, num_features = ndi.label(wt_mask) # scipy.ndimage
117 component_sizes = np.bincount(labeled.ravel()) # numpy
118 largest_label = component_sizes[1:].argmax() + 1
119 wt_mask = (labeled == largest_label)
120
121 # Closing (radius=4)
122 wt_mask = ndi.binary_closing(wt_mask, structure=se) # scipy.ndimage
123
124 # Fill holes
125 wt_mask = ndi.binary_fill_holes(wt_mask) # scipy.ndimage
126
127 # Constrain to brain mask
128 wt_mask = wt_mask & brain_mask
129
130 masks['WT'] = wt_mask
131
132 # -----
133 # STEP 2: Segment Enhancing Tumor (ET)
134 # -----
135
136 # Compute enhancement image
137 enh = t1ce - t1 # numpy
138
139 # Normalize enhancement to [0, 1]
140 enh_norm = (enh - enh.min()) / (enh.max() - enh.min() + 1e-8) #
    numpy
141
142 # Gamma correction
143 enh_gamma = adjust_gamma(enh_norm, gamma=0.5) # skimage.exposure
144
145 # Otsu thresholding (computed only on WT voxels)
146 threshold = threshold_otsu(enh_gamma[wt_mask]) # skimage.filters
147 et_mask = (enh_gamma > threshold) & wt_mask # numpy
148
149 # Boundary subtraction
150 boundary_thickness = 3
151 boundary = brain_mask & ~ndi.binary_erosion(brain_mask, structure=
    morphology.ball(boundary_thickness))
152 expanded_boundary_2 = ndi.binary_dilation(boundary, structure=
    morphology.ball(2)) & brain_mask
153 et_mask = et_mask & ~(expanded_boundary_2 & ~wt_mask) # Remove ET
    outside WT in boundary
154
155 # Hierarchy: ET must be within WT
156 et_mask = et_mask & wt_mask # numpy
157
158 # Remove small objects
159 et_mask = morphology.remove_small_objects(et_mask, min_size=500) #

```

```

160         skimage.morphology
161     masks['ET'] = et_mask
162
163     # -----
164     # STEP 3: Segment Necrotic Core (NCR)
165     # -----
166
167     # Use negative T1CE (necrosis appears dark)
168     ncr_data = -t1ce # numpy
169
170     # Growth constraint: NCR can grow in WT but not in ET
171     ncr_growth_constraint = wt_mask & ~et_mask # numpy
172
173     # Compute ET convex hull for seed constraint
174     if et_mask.sum() >= 10:
175         # Find connected components in ET
176         labeled_et, num_components = ndi.label(et_mask) # scipy.ndimage
177
178         hull_mask = np.zeros_like(et_mask, dtype=bool) # numpy
179
180         for component_id in range(1, num_components + 1):
181             component_mask = (labeled_et == component_id)
182             coords = np.column_stack(np.where(component_mask)) # numpy
183
184             # Check rank using SVD
185             coords_centered = coords - coords.mean(axis=0) # numpy
186             U, s, Vt = np.linalg.svd(coords_centered, full_matrices=
187                 False) # numpy.linalg
188             rank = np.sum(s > max(1e-6, s[0] * 1e-4)) # numpy
189
190             if rank == 3 and len(coords) >= 4:
191                 # Compute 3D convex hull
192                 hull = ConvexHull(coords, qhull_options='QJ') # scipy.
193                     spatial
194
195                 # Test points in bounding box
196                 z_min, z_max = coords[:, 0].min(), coords[:, 0].max() +
197                     1
198                 y_min, y_max = coords[:, 1].min(), coords[:, 1].max() +
199                     1
200                 x_min, x_max = coords[:, 2].min(), coords[:, 2].max() +
201                     1
202
203                 z, y, x = np.meshgrid(
204                     np.arange(z_min, z_max),
205                     np.arange(y_min, y_max),
206                     np.arange(x_min, x_max),
207                     indexing='ij'
208                 ) # numpy
209                 grid_points = np.column_stack([z.ravel(), y.ravel(), x.
210                     ravel()]) # numpy
211
212                 # Test if points are inside hull using Delaunay
213                 delaunay = Delaunay(hull.points[hull.vertices]) # scipy
214                     .spatial
215                 inside = delaunay.find_simplex(grid_points) >= 0 #
216                     scipy.spatial

```

```

209         component_hull = np.zeros_like(et_mask, dtype=bool) #
210         numpy
211         component_hull[z_min:z_max, y_min:y_max, x_min:x_max] =
212             inside.reshape(
213                 (z_max - z_min, y_max - y_min, x_max - x_min)
214             )
215         # Shrink hull (shrink_factor=0.8)
216         iterations = max(1, int((1.0 - 0.8) * 3))
217         for _ in range(iterations):
218             component_hull = ndi.binary_erosion(component_hull,
219                 structure=np.ones((3, 3, 3))) # scipy.ndimage
220
221         hull_mask |= component_hull
222     else:
223         # Use component directly for degenerate cases
224         hull_mask |= component_mask
225
226     et_convex_hull = hull_mask
227 else:
228     et_convex_hull = et_mask
229
230 # Seed constraint: convex hull within WT
231 ncr_seed_constraint = et_convex_hull & wt_mask # numpy
232
233 # Select foreground seeds by blocks
234 block_size = 2
235 n_seeds = 200
236 percentile_threshold = 30.0
237
238 # Divide image into blocks and compute mean intensity
239 block_stats = []
240 for z in range(0, ncr_data.shape[0], block_size):
241     for y in range(0, ncr_data.shape[1], block_size):
242         for x in range(0, ncr_data.shape[2], block_size):
243             block = ncr_data[z:z+block_size, y:y+block_size, x:x+
244                 block_size]
245             block_mask = wt_mask[z:z+block_size, y:y+block_size, x:x+
246                 block_size]
247             if block_mask.sum() > 0:
248                 mean_intensity = np.mean(block[block_mask]) # numpy
249                 threshold = np.percentile(ncr_data[wt_mask],
250                     percentile_threshold) # numpy
251                 if mean_intensity > threshold:
252                     block_stats.append({
253                         'mean': mean_intensity,
254                         'coords': (z, y, x, z+block_size, y+
255                             block_size, x+block_size)
256                     })
257
258 # Sort by mean intensity and select top n_seeds
259 block_stats.sort(key=lambda x: x['mean'], reverse=True)
260 selected_blocks = block_stats[:n_seeds]
261
262 # Create seed mask
263 seeds = np.zeros_like(ncr_data, dtype=bool) # numpy
264 for block in selected_blocks:

```

```

260     z, y, x, z_end, y_end, x_end = block['coords']
261     seeds[z:z_end, y:y_end, x:x_end] = True
262     seeds = seeds & wt_mask & ncr_seed_constraint # numpy
263
264     # Choose segmentation method (default: watershed)
265     ncr_method = 'watershed' # or 'seeded_region_growing'
266
267     if ncr_method == 'watershed':
268         # Convert foreground seeds to coordinates
269         fg_seed_coords = np.column_stack(np.where(seeds)) # numpy
270
271         # Select background seeds randomly from outside WT
272         bg_candidate_mask = brain_mask & ~wt_mask # numpy
273         bg_candidate_coords = np.column_stack(np.where(bg_candidate_mask
274                                                    )) # numpy
275
276         np.random.seed(42) # For reproducibility
277         if len(bg_candidate_coords) >= n_seeds:
278             indices = np.random.choice(len(bg_candidate_coords), size=
279                                       n_seeds, replace=False) # numpy
280             bg_seed_coords = bg_candidate_coords[indices]
281         else:
282             bg_seed_coords = bg_candidate_coords
283
284         # Normalize volume within brain mask
285         vals = ncr_data[brain_mask]
286         vmin, vmax = float(vals.min()), float(vals.max())
287         vol_n = (ncr_data - vmin) / (vmax - vmin + 1e-8) # numpy
288         vol_n = np.clip(vol_n, 0, 1) # numpy
289
290         # Create markers (1=BG, 2=FG)
291         markers = np.zeros_like(ncr_data, dtype=np.int32) # numpy
292         for r, c, z in bg_seed_coords:
293             markers[r, c, z] = 1
294         for r, c, z in fg_seed_coords:
295             markers[r, c, z] = 2
296
297         # Compute morphological gradient
298         se = morphology.ball(1) # skimage.morphology
299         dil = ndi.grey_dilation(vol_n, footprint=se) # scipy.ndimage
300         ero = ndi.grey_erosion(vol_n, footprint=se) # scipy.ndimage
301         grad = dil - ero # numpy
302
303         # Watershed segmentation
304         labels = watershed(image=grad, markers=markers, mask=brain_mask)
305         # skimage.segmentation
306
307         # Extract NCR (label 2 = foreground) and constrain to growth
308         region
309         ncr_mask = (labels == 2) & ncr_growth_constraint # numpy
310
311     else: # seeded_region_growing
312         # SRG with relative threshold
313         srg_threshold = 0.2 # Relative: |pixel - mean| < mean *
314                             threshold
315         max_ncr_size = int(wt_mask.sum() * 0.2)
316
317         # Initialize region with seeds

```

```

313     region = seeds.copy()
314     region_mean = np.mean(ncr_data[seeds]) # numpy
315
316     # Sequentially Sorted List (SSL) of candidate neighbors
317     ssl = []
318     processed = region.copy()
319
320     # Add seed neighbors to SSL
321     seed_points = np.argwhere(region) # numpy
322     for point in seed_points:
323         for neighbor in get_neighbors_26(point, ncr_data.shape):
324             if not processed[neighbor] and ncr_growth_constraint[
325                 neighbor]:
326                 intensity = ncr_data[neighbor]
327                 delta = abs(intensity - region_mean) # numpy
328                 ssl.append((delta, neighbor))
329                 processed[neighbor] = True
330
331     ssl.sort(key=lambda x: x[0]) # Sort by delta
332
333     # Main SRG loop
334     while ssl and region.sum() < max_ncr_size:
335         delta, candidate = ssl.pop(0)
336
337         # Relative threshold check: |pixel - mean| < mean *
338         # threshold
339         if abs(region_mean) < 1e-6:
340             should_add = delta < srg_threshold
341         else:
342             should_add = delta < abs(region_mean) * srg_threshold
343
344         if should_add:
345             region[candidate] = True
346             # Update region mean incrementally
347             n = region.sum()
348             region_mean = ((n - 1) * region_mean + ncr_data[
349                 candidate]) / n # numpy
350
351             # Add unprocessed neighbors to SSL
352             for neighbor in get_neighbors_26(candidate, ncr_data.
353                 shape):
354                 if not processed[neighbor] and ncr_growth_constraint
355                 [neighbor]:
356                     intensity = ncr_data[neighbor]
357                     delta = abs(intensity - region_mean) # numpy
358                     # Insert into SSL maintaining sorted order
359                     idx = 0
360                     while idx < len(ssl) and ssl[idx][0] < delta:
361                         idx += 1
362                     ssl.insert(idx, (delta, neighbor))
363                     processed[neighbor] = True
364
365     ncr_mask = region & ncr_growth_constraint # numpy
366
367     # Morphological operations
368     # Dilation (radius=1, iterations=2)
369     se = morphology.ball(1) # skimage.morphology
370     for _ in range(2):

```

```

366         ncr_mask = ndi.binary_dilation(ncr_mask, structure=se) # scipy.
            ndimage
367         ncr_mask = ncr_mask & ncr_growth_constraint # numpy
368
369     # Fallback if NCR too small (< 5000 voxels)
370     if ncr_mask.sum() < 5000:
371         # Convexify ET to get TC approximation
372         tc_from_et = convex_hull_mask(et_mask, shrink_factor=1.0,
            erosion_radius=0)
373         tc_from_et = tc_from_et & wt_mask
374         ncr_mask = tc_from_et & ~et_mask # NCR = TC - ET
375
376     masks['NCR'] = ncr_mask
377
378     # -----
379     # STEP 4: Derive Tumor Core (TC)
380     # -----
381
382     # Union of NCR and ET
383     tc_mask = masks['NCR'] | masks['ET'] # numpy
384
385     # Morphological operations
386     # Closing (radius=2)
387     se = morphology.ball(2) # skimage.morphology
388     tc_mask = ndi.binary_closing(tc_mask, structure=se) # scipy.ndimage
389
390     # Fill holes
391     tc_mask = ndi.binary_fill_holes(tc_mask) # scipy.ndimage
392
393     # Dilation (radius=2)
394     tc_mask = ndi.binary_dilation(tc_mask, structure=se) # scipy.
            ndimage
395
396     # Fill holes again
397     tc_mask = ndi.binary_fill_holes(tc_mask) # scipy.ndimage
398
399     masks['TC'] = tc_mask
400
401     # -----
402     # STEP 5: Derive Edema (ED)
403     # -----
404
405     # Subtraction: ED = WT - TC
406     ed_mask = wt_mask & ~tc_mask # numpy
407
408     masks['ED'] = ed_mask
409
410     # =====
411     # PHASE 4: EVALUATION (if ground truth available)
412     # =====
413
414     gt_path = patient_dir / f"{patient_id}_seg.nii.gz"
415     metrics = {}
416
417     if gt_path.exists():
418         gt_data, _, _ = load_nifti(gt_path) # nibabel
419
420         for region in ['WT', 'TC', 'ET', 'NCR', 'ED']:

```

```

421     pred_mask = masks[region]
422
423     # Extract ground truth labels
424     label_map = {
425         'NCR': [1],
426         'ED': [2],
427         'ET': [4],
428         'WT': [1, 2, 4],
429         'TC': [1, 4]
430     }
431     gt_mask = np.isin(gt_data, label_map[region]) # numpy
432
433     # Dice coefficient
434     intersection = (pred_mask & gt_mask).sum() # numpy
435     union = pred_mask.sum() + gt_mask.sum() # numpy
436     dice = 2 * intersection / (union + 1e-8) # numpy
437
438     # Hausdorff distance (95th percentile)
439     hd95 = calculate_hausdorff(pred_mask, gt_mask, percentile
440                               =95) # scipy.spatial.distance
441
442     metrics[region] = {'dice': dice, 'hausdorff_95': hd95}
443
444     # =====
445     # PHASE 5: SAVE OUTPUTS
446     # =====
447
448     if output_dir is not None:
449         patient_output = output_dir / patient_id
450         patient_output.mkdir(parents=True, exist_ok=True)
451
452         # Save segmentation masks as NIfTI files
453         _, affine, header = modalities['flair']
454         for region in ['WT', 'TC', 'ET', 'NCR', 'ED']:
455             output_path = patient_output / f"{patient_id}_{region}.nii.
456                               gz"
457             save_nifti(masks[region].astype(np.float32), affine, header,
458                       output_path) # nibabel
459
460         # Save metrics as JSON
461         if metrics:
462             metrics_path = patient_output / f"{patient_id}_metrics.json"
463             with open(metrics_path, 'w') as f:
464                 json.dump(metrics, f, indent=2) # json
465
466     return {
467         'predictions': masks,
468         'metrics': metrics,
469         'preprocessed': preprocessed
470     }

```